Relatório da Avaliação 2 - Teoria dos Grafos

Alunos: Yago Melo da Costa - 1631437

Introdução

Este relatório apresenta a implementação de uma biblioteca de grafos em Python. Foram desenvolvidas duas classes principais: Graph (grafos não direcionados) e Digraph (grafos direcionados), com suporte a operações fundamentais e algoritmos clássicos. A implementação foi validada com casos de teste, incluindo uma instância real da rede de estradas de Nova York (USA-road-d.NY.gr).

Explicação para o **Grafo**:

A classe Graph foi desenvolvida para representar grafos genéricos não direcionados, oferecendo suporte tanto a grafos ponderados quanto não ponderados. Esta classe estrutura-se em torno de três componentes principais: um dicionário adj para armazenar as listas de adjacência, que mapeia cada vértice aos seus vizinhos; um dicionário degrees que mantém o grau de cada vértice, atualizado dinamicamente conforme arestas são adicionadas; e um dicionário weights utilizado exclusivamente para armazenar os pesos das arestas em grafos ponderados. Além dessas estruturas fundamentais, a classe incorpora métodos abrangentes para adicionar vértices e arestas, calcular propriedades essenciais do grafo e executar algoritmos clássicos da teoria dos grafos.

Quanto às decisões de implementação, optou-se por uma representação baseada em listas de adjacência, utilizando dicionários para garantir acesso eficiente aos vértices e suas arestas. Essa abordagem permite consultas rápidas e interação simplificada sobre os vizinhos de qualquer vértice, otimizando operações comuns. Para os algoritmos
fundamentais, implementaram-se quatro rotinas principais. A busca em largura (BFS) emprega uma fila para explorar os vértices em ordem crescente de distância a partir de um vértice de origem, retornando tanto as distâncias quanto os predecessores. A busca em profundidade (DFS) segue uma

abordagem recursiva, atribuindo timestamps de entrada e saída a cada vértice visitado, o que permite reconstruir a árvore de busca e analisar a estrutura do grafo. O algoritmo de Bellman-Ford (BF) itera sobre todas as arestas repetidamente para encontrar os caminhos mínimos a partir de uma origem, com capacidade adicional de detectar ciclos negativos. Por fim, o algoritmo de Dijkstra utiliza uma fila de prioridade baseada em heap para calcular caminhos mínimos de forma eficiente em grafos sem pesos negativos, garantindo desempenho otimizado mesmo em estruturas de maior escala. Esta implementação busca equilibrar clareza conceitual, eficiência computacional e fidelidade às especificações dos algoritmos originais.

1. Resultados do teste para um “Graph” criado

```
Número de nós: 4
Número de arestas: 5
Vizinhos de 2: {1, 3, 4}
Grau de 2: 3
Peso da aresta (1, 2): 4
Menor grau: 2
Maior grau: 3
Distâncias BFS a partir de 1: {1: 0, 2: 1, 3: 1, 4: 2}
Predecessores BFS a partir de 1: {1: None, 2: 1, 3: 1, 4: 2}
Predecessores DFS a partir de 1: {1: None, 2: 1, 3: 2, 4: 3}
Tempos de início: {1: 1, 2: 2, 3: 3, 4: 4}
Tempos de término: {4: 5, 3: 6, 2: 7, 1: 8}
Distâncias Bellman-Ford a partir de 1: {1: 0, 2: 3, 3: 1, 4: 8}
Predecessores Bellman-Ford a partir de 1: {1: None, 2: 3, 3: 1, 4: 2}
Distâncias Dijkstra a partir de 1: {1: 0, 2: 3, 3: 1, 4: 8}
Predecessores Dijkstra a partir de 1: {1: None, 2: 3, 3: 1, 4: 2}
```

Explicação para o **Dígrafo:**

A classe Digraph foi concebida para modelar grafos direcionados, oferecendo suporte à representação de grafos ponderados através de uma estrutura especializada. Esta implementação inclui quatro componentes principais: um dicionário adj que armazena as listas de adjacência de saída, mapeando cada vértice aos seus sucessores; um dicionário complementar in_adj que registra os predecessores de cada vértice, capturando assim as arestas de entrada; um dicionário degrees que mantém o controle do grau total (entrada + saída) de cada vértice; e um dicionário weights dedicado ao armazenamento dos pesos das arestas quando aplicável. Além dessas estruturas fundamentais, a classe réplica a interface funcional da classe Graph, incorporando métodos completos para adição de vértices e arcos, cálculo de propriedades essenciais e execução dos principais algoritmos de teoria dos grafos adaptados à natureza direcionada dos dígrafos.

Nas decisões de implementação, adotou-se uma abordagem simétrica à representação de arestas de entrada e saída, utilizando dicionários para garantir acesso eficiente tanto aos predecessores quanto aos sucessores de qualquer vértice. Essa dualidade estrutural permite consultas rápidas e interação simplificada sobre a vizinhança completa de um nó, otimizando operações como cálculo de grau e navegação no grafo. Para os algoritmos fundamentais, implementaram-se quatro rotinas principais adaptadas à direcionalidade das arestas. A busca em largura (BFS) emprega uma fila tradicional para explorar os vértices em ordem crescente de distância a partir de uma origem. A busca em profundidade (DFS) segue uma implementação recursiva que atribui timestamps de entrada e saída durante o percurso, essencial para análise de dependências. O algoritmo de Bellman-Ford (BF) itera exaustivamente sobre todos os arcos para calcular caminhos mínimos mesmo na presença de pesos negativos, enquanto o algoritmo de Dijkstra utiliza uma fila de prioridade baseada em heap para eficiência superior em grafos sem pesos negativos. Esta arquitetura mantém consistência conceitual com a implementação de grafos não direcionados enquanto respeita as particularidades topológicas dos dígrafos.

2. Resultados do teste para o arquivo “USA-road-d.NY.gr” fornecido

